EMBODIED AGENT STORYTELLING REPORT

Tracking Agent

不是“会框人”，而是“会持续理解任务”的机器人运行基座

如果只看算法，这像一个 tracking 系统；如果站在系统设计角度看，它其实是在回答一个更大的问题：机器人怎样把“持续看世界”和“理解人类任务”接成一条真正可用的链。

3+1
推荐运行形态

1
状态真相源

Pi + Direct
决策路径

Tracking
成熟能力样板

Chat-first | Perception 常驻 | Single Runner | Session Truth | Skill-Pluggable

WHY THE PROJECT EXISTS

为什么普通 tracking demo 不够？

它能框人，但不一定能理解任务
用户说的是“跟着最开始出现的穿黑衣服的人”，不是“跟踪 ID 7”。系统必须先理解语义，再执行。

它能追一帧，但不一定能维持连续状态
真正的机器人不是处理单帧图像，而是要跨多轮、跨时间地记住“我现在正在跟谁”。

它能显示画框，但不一定能解释自己在做什么
如果没有统一状态与展示层，用户看到的只是框，看不到系统为什么这样选、下一步准备做什么。

真正难的，不是某个模型强不强，而是整条链必须连起来。

- 机器人能不能一直看着现实世界？
- 用户能不能用自然语言告诉它“去跟着那个人”？
- 系统能不能把“那个人是谁”记住，而不是每一轮都重新猜？
- 当目标短暂丢失、重新出现、或用户要求切换时，系统能不能继续工作？

THE REAL GOAL

这个项目真正想解决什么？

持续感知
机器人一直在看世界，不需要每次等用户开口才“醒过来”。

对话驱动的任务理解
用户用自然语言发任务，系统把当前世界状态和历史状态一起拿来解释这句话。

单一状态真相源
系统必须稳定记住“我正在跟谁、上一步发生了什么”，而不是让多个缓存互相猜。

能力可扩展
tracking 只是第一能力样板，speech、web_search 乃至更多 embodied skill 都要能自然接进来。

它想解决的不是“目标检测准不准”这一个点，而是怎样把 perception、对话理解、状态持久化、技能调用和展示层，收敛成同一个 agent runtime。

CORE THESIS

这套方案最聪明的地方：不让任何一个模块变成“整个系统”

Perception 只负责“看见”
它持续读 camera / video、产出 observation、保存 keyframe 和 snapshot，但不替系统做高层决策。

Runner 只负责“这一轮”
每来一个 turn，就认真处理完这一轮：读状态、组上下文、选 skill 路径、落盘结果。

Session state 只有一个主真相源
session.json 把“我正在跟谁、上一轮结果是什么、skill 记住了什么”稳定收在一个地方。

Skill 是能力模块，不是系统分叉
tracking、speech、web_search 都通过统一 surface 接入，backend 主干不因新能力而变形。

Viewer 只负责让人看懂
viewer 是 read-only projection layer，负责显示系统状态，而不是控制业务逻辑。

如果把整套系统压缩成一张图

□□□□□camera / video□
□
[Perception Service]
□□□ detection / tracking / snapshot
□
[Persisted World State]
□□□□□□□□□snapshot
□
□□ / loop / script □□□□ turn
□
[PiAgentRunner]
□□ session + perception□□□ route context
□
[Skill Execution]
tracking / speech / web_search
□
[Session Truth]
latest_result / skill_cache / history
□
[Viewer Stream]
□□□□□□□□□□
□

五个架构关键词

Chat-first | Perception 常驻 | Single Runner | Session Truth | Skill-Pluggable | Viewer 只读

Chat-first, not perception-first
turn 由聊天、脚本或 loop 事件触发；perception 负责提供 grounded context。

Perception is the only always-on subsystem
只有 perception 常驻运行；其余组件围绕一次次 turn 工作。

Single runner path
所有 turn 最终都收敛到 PiAgentRunner 这一条主处理链。

Single persisted state truth
session.json 是 agent-owned state 的主真相源。

Skills are ordinary modules
tracking、speech、web_search 都通过统一 skill surface 接入。

TELL IT LIKE A STORY

用一个真实场景，讲完整套 embodied agent 流程

场景：用户说“开始跟踪最开始出现的穿黑衣服的人。”

1. **系统先看到世界**
   Perception 服务一直在跑，不需要等用户下命令。它像机器人的眼睛，持续把“现在看到了什么”写成 snapshot。

2. **用户给出的是语义任务，不是 track id**
   “最开始出现的穿黑衣服的人”是一句带上下文、带歧义空间的话。系统不能直接执行，它要先理解。

3. **Runner 组装这次 turn 的完整上下文**
   它把最近对话、当前世界和已有 session 状态拼起来。于是系统不是“听一句话”，而是“带着记忆去理解一句话”。

4. **该自由发挥的地方让模型发挥，该收紧的地方就收紧**
   tracking init / track 这种 fragile workflow，优先走 deterministic entry script，而不是完全依赖开放式 LLM 推理。

5. **目标一旦确认，就正式写进 session**
   系统不是“这轮猜到是谁了”，而是“从这一刻起，我正式知道自己正在跟谁”。

6. **Tracking loop 负责持续推进，但不是系统总控**
   它只在 tracking 这个能力上推动 continuation / recovery，不把整个 runtime 拖进一个大 while loop。

7. **Viewer 负责把系统脑子里的想法展示出来**
   用户能看到目标框、记忆、对话和当前状态；系统因此变得可解释、可观察、可调试。

CAST LIST

主要组件，换成“角色表”来讲

角色	代表路径	它在故事里扮演什么角色
眼睛	backend/perception/	持续看世界，把 observation、snapshot、keyframe 写下来
单轮工作台	backend/agent/runner.py	处理一次 turn：读状态、组上下文、选路径、落结果
决策边界	backend/agent/pi_protocol.py	规定 Pi 能读什么、该返回什么
记忆本	backend/persistence/ + session.json	把“系统当前正在做什么”稳稳记住
技能面板	backend/skills.py	让 tracking / speech / web_search 以统一方式接进来
主力能力	skills/tracking/	做目标初始化、持续跟踪、记忆更新与 viewer 模块
展示层	agent_viewer_stream + tracking-viewer	把系统状态翻译成人类可读的 UI
舞台监督	scripts/	把 perception、loop、viewer、frontend 分别启动起来

为什么这种分工舒服？

因为你能一眼看出谁负责什么
一套系统只要角色分工清楚，它就更容易维护，也更容易解释给别人听。

因为新增能力不会逼 backend 变形
新 skill 往 skills/<name>/ 里接，不需要把主干 runtime 再拆一遍。

因为系统主线永远只有一条
不是 perception 一套逻辑、loop 一套逻辑、前端一套逻辑各说各话，而是统一回到 runner 和 session truth。

部署方式也很“人能理解”

Perception
一直看世界，负责把现实变成机器可读状态。

Tracking Loop
在 tracking 语境里推动 continuation / recovery。

Viewer Stream + Frontend
把系统状态变成用户可见的界面。

“3 个长期进程 + 1 个可选前端”的部署方式不花哨，但特别实用：职责分离、定位清楚、前端可选、单机易跑。

TRADEOFFS

这套方案的成熟，不在于没有边界，而在于边界讲得清楚

要诚实地讲它的边界

它更像 embodied decision kernel，不是完整 motion stack
现在更强的是“看、记、理解、执行 capability”的闭环，而不是完整机器人运动控制闭环。

它非常适合单机 / 单 session，但还不是分布式多机系统
本地文件共享状态对当前阶段特别合理，但未来跨设备部署需要更明确的 service / bus 边界。

tracking 是当前最成熟的 embodied capability
这不是短板，反而证明项目在先把一个核心能力打磨到位，而不是空泛许愿。

最值得肯定的设计取舍

不把 perception 变成大脑
感知层长期跑，但不越权做高层决策。

不把 tracking loop 变成总控中心
loop 只是 tracking 能力的推进器，不绑架整个 runtime 形状。

不让 LLM 包办脆弱流程
tracking init / track 走 deterministic entry script，这是一种成熟的工程选择。

不让 viewer 决定系统怎么运行
viewer 只负责让人看懂，不反向塑造业务逻辑。

这套架构真正成熟的地方，不在于它“什么都做”，而在于它知道哪些事情现在该做，哪些事情暂时不要装成已经做好了。

OUTLOOK

Future Work，以及你上台时最应该怎么讲

下一步最值得长什么？

更强的部署韧性
补齐 perception、loop、viewer 的健康检查、恢复与异常处理，让它更像持续运行系统。

更多 embodied capabilities
沿着现有 skill surface 接入 spatial QA、action recommendation、multimodal memory 等新能力。

更强的 observability
系统化记录 turn latency、route decision、rewrite worker 成功率和 recovery 质量。

远程 / 多设备部署
当跨机部署成为刚需时，再把当前清晰边界升级成真正的 service 边界。

如果拿去讲，我建议这样收尾

我们做的不是一个 tracking demo，而是一套让机器人把‘看见世界’和‘理解任务’接起来的 embodied agent runtime。

先讲问题
普通 tracking demo 只能框人，不能形成持续任务闭环。

再讲架构
perception 常驻、runner 单轮处理、session 单源、skills 插拔、viewer 只读。

最后讲价值
这不是一次性脚本堆叠，而是一套可以继续长能力的 embodied agent kernel。

最后压缩成五个关键词

持续感知 | 单轮决策主链 | 单一状态真相源 | 能力可插拔 | 系统可观察

这五个词，基本就是这套 embodied agent 技术方案的骨架。也是你讲完之后，听众应该带走的记忆点。